



TRƯỜNG ĐẠI HỌC NGÂN HÀNG
THÀNH PHỐ HỒ CHÍ MINH

LẬP TRÌNH WEB

CHƯƠNG 3 NGÔN NGỮ LẬP TRÌNH PHP

1. Giới thiệu
2. Biến, kiểu dữ liệu và toán tử
3. Cấu trúc điều kiện
4. Cấu trúc lặp
5. Hàm
6. Nhúng PHP vào HTML

NGÔN NGỮ LẬP TRÌNH PHP

- PHP (PHP: Hypertext PreProcessor) được tạo ra bởi Rasmus Lerdorf và được công bố lần đầu vào năm 1995, là một ngôn ngữ kịch bản (thông dịch) miễn phí.
- Được sử dụng kết hợp với HTML, CSS, Javascript để tạo ra các website động.
- Có thể chạy trên nhiều hệ điều hành: Windows, Linux, Unix, MacOS, . . . và hỗ trợ kết nối với nhiều hệ quản trị CSDL: Oracle, SQL Server, MySQL, MongoDB, . . .

TẬP TIN PHP

- Nội dung trong một tập tin PHP gồm văn bản, HTML, CSS, Javascript và các đoạn mã PHP.
- Các đoạn mã PHP được thực thi, xử lý tại máy chủ (server-side) và kết quả được trả về cho trình duyệt dưới dạng HTML thuần túy.
- Tập tin PHP có phần mở rộng là **.php**

ƯU ĐIỂM VÀ NHƯỢC ĐIỂM CỦA PHP

▪ Ưu điểm:

- Dễ học, dễ sử dụng.
- Có cộng đồng hỗ trợ lớn.
- Cung cấp nhiều framework và thư viện hỗ trợ.

▪ Nhược điểm:

- Việc bảo vệ mã nguồn còn hạn chế.
- Chỉ hỗ trợ xây dựng các ứng dụng web.

MỘT SỐ ĐẶC ĐIỂM CỦA PHP

- Không cần kiểu dữ liệu khi thực hiện khai báo biến, biến sẽ nhận dạng kiểu dữ liệu dựa vào giá trị được gán.
- Tên biến bắt buộc phải bắt đầu bằng ký tự \$.
- Các đoạn mã PHP đặt trong cặp thẻ:

<?php

//Các đoạn mã PHP

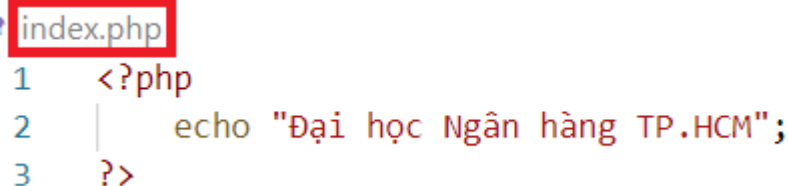
?>

- Sử dụng dấu **//** hoặc **#** để chú thích một dòng và cặp dấu **/* */** để chú thích nhiều dòng.

CHẠY CHƯƠNG TRÌNH PHP ĐƠN GIẢN

- Sử dụng các trình soạn thảo mã nguồn (Visual Studio Code, notepad++) hoặc các IDE khác và đặt vào các đoạn mã PHP trong cặp thẻ `<?php ?>` → Lưu file với phần mở rộng là **.php**

Tên file với phần
mở rộng là **.php**

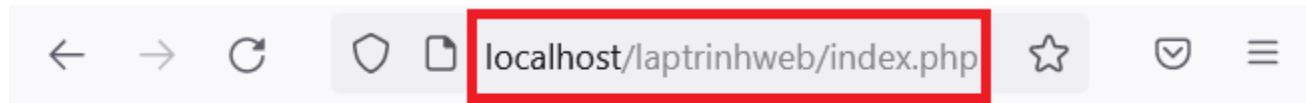


```
index.php
1  <?php
2      echo "Đại học Ngân hàng TP.HCM";
3  ?>
```

- Đặt file *.php trong thư mục: “<đường dẫn thư
mục cài đặt Wampp>/www”

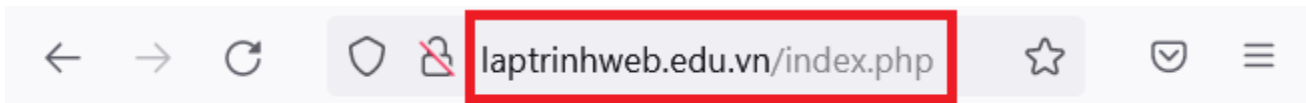
CHẠY CHƯƠNG TRÌNH PHP ĐƠN GIẢN

- Mở trình duyệt và chạy chương trình



Đại học Ngân hàng TP.HCM

Chạy file mã nguồn [index.php](#) được đặt trong thư mục [laptrinhweb](#)



Đại học Ngân hàng TP.HCM

Chạy file mã nguồn [index.php](#) với Virtual Host

BIẾN

- Biến đại diện cho một vùng nhớ của máy tính và là nơi lưu trữ các dữ liệu mà chương trình sử dụng trong quá trình thực thi.
- PHP không yêu cầu chỉ định kiểu dữ liệu khi khai báo biến. Dựa vào giá trị được gán, biến sẽ nhận dạng kiểu dữ liệu.
- Cần chú ý vị trí khai báo và sử dụng biến để tránh các lỗi logic trong quá trình lập trình.

QUY TẮC ĐẶT TÊN BIẾN

- Tên biến phải bắt đầu bằng dấu \$, theo sau là một ký tự chữ cái alphabet hoặc ký tự _

Ví dụ: **\$ten**, **\$_name**

- Tên biến chỉ chứa các ký tự a-z, A-Z, 0-9, _
- Tên biến không được có khoảng trắng, trường hợp tên biến có nhiều từ có thể sử dụng dấu _ để ngăn cách giữa các từ.

Ví dụ: **\$ho_ten**

- Tên biến phân biệt chữ hoa, thường.

QUY TẮC ĐẶT TÊN BIẾN

- Các tên biến hợp lệ: `$ten`, `$_name`, `$x1`, `$Ho_Ten`
- Các tên biến không hợp lệ: `$1x`, `name`, `$Ho Ten`
- Tên biến có phân biệt hoa thường:

`$hoten` khác với `$HoTen`

`$diem_so` khác với `$Diem_So`

HẲNG

- Hằng có chức năng tương tự như biến dùng để lưu trữ một giá trị.
- Giá trị của hằng không thể thay đổi được trong suốt quá trình thực thi của chương trình.
- Sử dụng hàm **define** hoặc từ khóa **const** để khai báo hằng
- Tên hằng không cần bắt đầu bằng dấu \$ và bắt đầu bằng một chữ cái hoặc dấu gạch dưới (_).

HẲNG

//Khai báo hằng sử dụng hàm define

```
define("SITE_NAME", "PHP Website");
```

//Khai báo hằng sử dụng hàm từ khóa const

```
const MAX_UPLOAD_SIZE = 1048576;
```

//Xuất ra giá trị của hằng SITE_NAME

```
echo SITE_NAME;
```

```
echo "<br>;"
```

//Xuất ra giá trị của hằng MAX_UPLOAD_SIZE

```
echo MAX_UPLOAD_SIZE;
```

KIỂU DỮ LIỆU

- Mặc dù không cần kiểu dữ liệu khi khai báo biến nhưng biến sẽ tự nhận dạng kiểu dữ liệu dựa vào giá trị được gán.
- Một số kiểu dữ liệu trong PHP:
 - Kiểu chuỗi (String)
 - Kiểu số nguyên (Integer)
 - Kiểu số thực (Float)
 - Kiểu luận lý (Boolean)
 - Kiểu mảng (Array)
 - Kiểu đối tượng (Object)
 - Kiểu NULL

KIỂU DỮ LIỆU STRING

- Chuỗi được đặt trong cặp dấu " " hoặc ' '.

Lệnh **echo** dùng để xuất dữ liệu ra màn hình

```
<?php  
$x = "Đại học Ngân hàng TP.HCM";
```

```
$y = 'Đại học Ngân hàng TP.HCM';
```

```
echo $x;
```

```
echo "<br>";
```

```
var_dump($y);  
?>
```

Lệnh **var_dump** dùng để xuất cấu trúc của biến ra màn hình gồm **kiểu dữ liệu** và **giá trị** của biến

KIỂU DỮ LIỆU SỐ

- Integer

```
<?php
    $year = 2022;
    var_dump($year);
?>
```

- Float

```
<?php
    $x = 10.365;
    var_dump($x);
?>
```


KIỂU DỮ LIỆU BOOLEAN

```
<?php
```

```
$x = true;
```

```
$y = false;
```

```
echo $x;
```

```
echo "<br>";
```

```
var_dump($y);
```

```
?>
```

KIỂU DỮ LIỆU MẢNG (ARRAY)

- Kiểu mảng cho phép lưu trữ nhiều giá trị trong một biến và các giá trị có thể có các kiểu dữ liệu khác nhau.
- Kiểu mảng có thể chia thành ba loại:
 - Mảng chỉ số (indexed array)
 - Mảng kết hợp (associative array)
 - Mảng đa chiều (multidimensional array).
- Trong PHP, hàm **array()** hoặc cặp dấu **[]** (từ phiên bản PHP 5.4) được sử dụng để tạo một biến mảng.

MẢNG CHỈ SỐ

- Mảng chỉ số sử dụng các chỉ số là số nguyên để truy cập các phần tử.

- Ví dụ:

Chỉ số

0

↓

1

↓

2

↓

```
<?php
$cars = array("Volvo", "BMW", "Toyota");
var_dump($cars);
echo $cars[0]; //Volvo
echo $cars[1]; //BMW
echo $cars[2]; //Toyota
?>
```

- Trường hợp không định nghĩa chỉ số cho mảng thì mặc định chỉ số bắt đầu từ **0**

MẢNG CHỈ SỐ

- Có thể sử dụng cặp dấu `[ ]` để khai báo mảng.
- Ví dụ:

Chỉ số

0

↓

1

↓

2

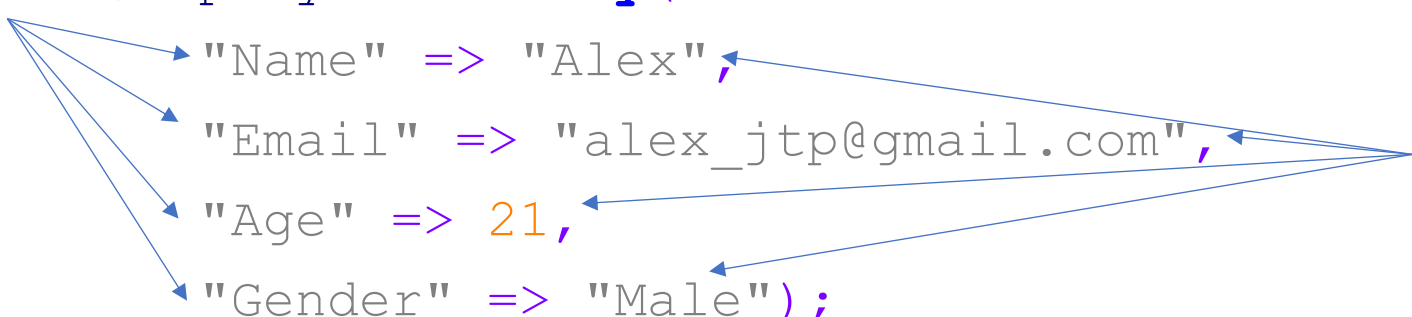
↓

```
<?php
$cars = ["Volvo", "BMW", "Toyota"];
var_dump($cars);
echo $cars[0]; //Volvo
echo $cars[1]; //BMW
echo $cars[2]; //Toyota
?>
```

MẢNG KẾT HỢP

- Mảng kết hợp sử dụng các khóa (key) dạng chuỗi để xác định và truy cập các giá trị.
- Các phần tử trong mảng kết hợp sẽ có dạng cặp "khóa" => "giá trị".

```
<?php
Khóa (Key)   $employee = array(
               "Name" => "Alex",
               "Email" => "alex_jtp@gmail.com",
               "Age"  => 21,
               "Gender" => "Male");
               Giá trị (Value)
echo $employee["Name"] . "<br>"; //Alex
echo $employee["Age"]; //21
?>
```



MẢNG ĐA CHIỀU

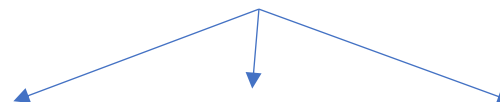
- Mảng đa chiều là một cấu trúc dữ liệu thường được sử dụng trong ngôn ngữ PHP.
- Mảng đa chiều là một mảng gồm một hoặc nhiều mảng con.
- Tùy vào số chiều của mảng mà sử dụng số lượng chỉ số hoặc khóa để truy cập vào các phần tử của mảng.
- Mảng hai chiều là loại mảng đa chiều thường được sử dụng nhất. Mảng hai chiều có thể xem như là một mảng gồm nhiều phần tử và mỗi phần tử là mảng một chiều.

MẢNG ĐA CHIỀU

Mảng **một** chiều, mỗi phần tử là
một **giá trị đơn**

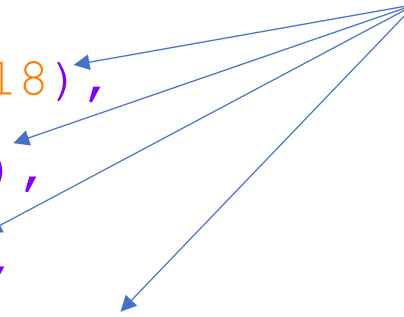
- Ví dụ:

```
$cars = array("Volvo", "BMW", "Toyota");
```



Mảng **hai** chiều, mỗi phần tử là
một **mảng một chiều**

```
$cars = array (  
    array("Volvo", 22, 18),  
    array("BMW", 15, 13),  
    array("Saab", 5, 2),  
    array("Land Rover", 17, 15)  
);
```



- Truy cập mảng 2 chiều:

```
echo $cars[0][0].": In stock: ".$cars[0][1].", sold: ".$cars[0][2].".<br>;
```

MẢNG ĐA CHIỀU

- Ví dụ: Mảng hai chiều gồm các mảng kết hợp

```
$products = array(  
    'paper' => array(  
        'copier' => "Copier & Multipurpose",  
        'inkjet' => "Inkjet Printer",  
        'laser' => "Laser Printer",  
        'photo' => "Photographic Paper"),  
    'pens' => array(  
        'ball' => "Ball Point",  
        'hilite' => "Highlighters",  
        'marker' => "Markers"),  
    'misc' => array(  
        'tape' => "Sticky Tape",  
        'glue' => "Adhesives",  
        'clips' => "Paperclips")  
);
```

- Truy cập mảng 2 chiều:

```
echo $products["paper"]["laser"]; //Laser Printer
```


MẢNG ĐA CHIỀU

- Ví dụ: Sử dụng cặp dấu `[ ]` tạo mảng hai chiều

```
$arrCourses = array ();
```

```
$arrCourses[101] = [  
    'name' => "Thiết kế web",  
    'credits' => "3"
```

```
];
```

```
$arrCourses[201] = [  
    'name' => "Lập trình web",  
    'credits' => "4"
```

```
];
```

```
echo $arrCourses[201]['name']; //Lập trình web
```

Từ phiên bản PHP 5.4 có thể sử dụng dấu `[ ]` để tạo mảng

KIỂU DỮ LIỆU NULL

- Kiểu NULL là kiểu dữ liệu đặc biệt.
- Một biến khi không được khởi tạo giá trị sẽ được gán giá trị là NULL.

```
<?php
```

```
$x = "Hello";
```

```
$x = null;
```

```
var_dump ($x) ;
```

```
?>
```

CHUYỂN ĐỔI KIỂU DỮ LIỆU

Cú pháp	Mô tả	Ví dụ
(int) (integer)	Chuyển sang kiểu số nguyên	<pre><?php \$x = "1"; \$y = "1.5"; \$z = (integer) \$x; \$t = (int) \$y; var_dump(\$z); echo "
"; var_dump(\$t); ?></pre>
(float) (double)	Chuyển sang kiểu số thực	<pre><?php \$x = "1.5"; \$y = "1.7"; \$z = (float) \$x; \$t = (double) \$y; var_dump(\$z); echo "
"; var_dump(\$t); ?></pre>

CHUYỂN ĐỔI KIỂU DỮ LIỆU

Cú pháp	Mô tả	Ví dụ
(bool) (boolean)	Chuyển sang kiểu số boolean	<pre><?php \$x = "1"; \$y = "0"; \$z = (bool) \$x; \$t = (boolean) \$y; var_dump(\$z); echo "
"; var_dump(\$t); ?></pre>
(string)	Chuyển sang kiểu chuỗi	<pre><?php \$x = 1; \$z = (string) \$x; \$t = (array) \$x; var_dump(\$z); echo "
"; var_dump(\$t); ?></pre>
(array)	Chuyển sang kiểu mảng	

TOÁN TỬ

- Được sử dụng để thực hiện các phép toán trên các biến và các giá trị.
- Trong PHP toán tử có thể chia thành các nhóm:
 - Toán tử toán học
 - Toán tử gán
 - Toán tử so sánh
 - Toán tử luận lý
 - Toán tử tăng/giảm
 - Toán tử trên chuỗi
 - Các toán tử khác

TOÁN TỬ TOÁN HỌC

Toán tử	Mô tả	Ví dụ
+	Cộng hai toán hạng	$\$x + \y
-	Trừ hai toán hạng	$\$x - \y
*	Nhân hai toán hạng	$\$x * \y
/	Chia hai toán hạng	$\$x / \y
%	Lấy phần dư của phép chia nguyên	$\$x \% \y
**	Lũy thừa	$\$x ** \$y (x^y)$

TOÁN TỬ GÁN

Toán tử	Ví dụ
=	$\$x = 5$ (gán giá trị 5 cho biến $\$x$)
+=	$\$x += 3$ tương đương $\$x = \$x + 3$
-=	$\$x -= 3$ tương đương $\$x = \$x - 3$
*=	$\$x *= 3$ tương đương $\$x = \$x * 3$
/=	$\$x /= 3$ tương đương $\$x = \$x / 3$
%=	$\$x \% = 3$ tương đương $\$x = \$x \% 3$

TOÁN TỬ SO SÁNH

Toán tử	Mô tả	Ví dụ	Kết quả
<code>==</code>	Kiểm tra bằng nhau	<code>\$x == \$y</code>	Trả về True nếu \$x bằng \$y (\$x và \$y có thể khác kiểu dữ liệu)
<code>===</code>	Kiểm tra bằng nhau đồng nhất	<code>\$x === \$y</code>	Trả về True nếu \$x bằng \$y và \$x cùng kiểu dữ liệu với \$y
<code>!= (<>)</code>	Kiểm tra khác nhau	<code>\$x != \$y</code>	Trả về True nếu \$x không bằng \$y
<code>!==</code>	Kiểm tra không đồng nhất	<code>\$x !== \$y</code>	Trả về True nếu \$x không bằng \$y hoặc \$x khác kiểu dữ liệu với \$y

TOÁN TỬ SO SÁNH

Toán tử	Mô tả	Ví dụ	Kết quả
$>$	Kiểm tra lớn hơn	$\$x > \y	Trả về True nếu $\$x$ lớn hơn $\$y$
$<$	Kiểm tra nhỏ hơn	$\$x < \y	Trả về True nếu $\$x$ nhỏ hơn $\$y$
$\geq$	Kiểm tra lớn hơn hoặc bằng	$\$x \geq \y	Trả về True nếu $\$x$ lớn hơn hoặc bằng $\$y$
$\leq$	Kiểm tra nhỏ hơn hoặc bằng	$\$x \leq \y	Trả về True nếu $\$x$ nhỏ hơn hoặc bằng $\$y$

TOÁN TỬ TĂNG/GIẢM

Toán tử	Mô tả	Ví dụ	Kết quả
++\$x	Tăng giá trị biến \$x lên một đơn vị và trả về kết quả	$x = 0;$ $t = ++x;$	$t = 1;$
\$x++	Trả về giá trị \$x và tăng giá trị biến \$x lên một đơn vị	$x = 0;$ $t = x++;$	$t = 0;$
--\$x	Giảm giá trị biến \$x xuống một đơn vị và trả về kết quả	$x = 0;$ $t = --x;$	$t = -1;$
\$x--	Trả về giá trị \$x và giảm giá trị biến \$x xuống một đơn vị	$x = 0;$ $t = x--;$	$t = 0;$

TOÁN TỬ LOGIC

Toán tử	Mô tả	Ví dụ
&&	Trả về kết quả là True khi cả hai biểu thức đều True	<pre><?php \$x = 10; \$y = 5; if (\$x == 10 && \$y == 5) echo "Hello world!"; ?></pre>
	Trả về kết quả là True nếu ít nhất một trong biểu thức là True	<pre><?php \$x = 10; \$y = 8; if (\$x == 10 \$y == 5) echo "Hello world!"; ?></pre>
!	Đảo ngược giá trị của biểu thức, từ True sang False và ngược lại	<pre><?php \$x = 8; if (!(\$x == 7)) echo "Hello world!"; ?></pre>

TOÁN TỬ LOGIC

Toán tử	Mô tả	Ví dụ
and	Trả về kết quả là True khi cả hai biểu thức đều True	<pre><?php \$x = 10; \$y = 5; if (\$x == 10 and \$y == 5) echo "Hello world!"; ?></pre>
or	Trả về kết quả là True nếu một trong hai biểu thức là True	<pre><?php \$x = 10; \$y = 8; if (\$x == 10 or \$y == 5) echo "Hello world!"; ?></pre>
xor	Trả về kết quả là True khi hai biểu thức có kết quả khác nhau	<pre><?php \$x = 10; \$y = 5; if (\$x == 10 xor \$y == 5) echo "Hello world!"; else echo "ĐHNNH"; ?></pre>

TOÁN TỬ TRÊN CHUỖI

Toán tử	Mô tả	Ví dụ
.	Nối chuỗi	<pre><?php \$text1 = "Hello "; \$text2 = "world"; \$text = \$text1.\$text2; ?></pre>
.=	$\$txt1 .= \$txt2$ tương đương $\$txt1 = \$txt1. \$txt2$	<pre><?php \$text1 = "Hello "; \$text2 = "world"; \$text1 .= \$text2; ?></pre>

TOÁN TỬ KHÁC

- Toán tử 3 ngôi
- Cú pháp:

<biểu thức> ? <giá trị 1> : <giá trị 2>

- Nếu <biểu thức> là True thì trả về <giá trị 1>, ngược lại trả về <giá trị 2>
- Ví dụ:

```
<?php
```

```
$x = 5;
```

```
$y = 10;
```

```
$max = $x > $y ? $x : $y;
```

```
?>
```

ĐỘ ƯU TIÊN CỦA TOÁN TỬ

Toán tử	Mức độ ưu tiên
**	1
++ --	2
!	3
* / %	4
+ - .	5
== != === !== <>	6
&&	7
	8
?:	9
= += -= *= /= %=	10
and	11
xor	12
or	13

Câu lệnh: **if**

- Cú pháp:

```
if (<biểu thức điều kiện>) {  
    <các câu lệnh nếu biểu thức điều kiện đúng>  
}
```


Câu lệnh: if

■ Ví dụ:

```
<?php
    $x = 15; $y = 10;
    if($x > $y){
        echo "Giá trị lớn hơn là ".$x;
    }
?>
```

```
<?php
    $x = 15; $y = 10; $z = 5;
    if($x > $y && $x > $z)
    {
        echo "Giá trị lớn nhất là ".$x;
    }
?>
```

Câu lệnh: **if - else**

- Cú pháp:

```
if (<biểu thức điều kiện>) {  
    <các câu lệnh nếu biểu thức điều kiện đúng>  
}  
else  
{  
    <các câu lệnh nếu biểu thức điều kiện sai>  
}
```

Câu lệnh: if - else

- Ví dụ:

```
<?php
    $x = 15; $y = 10;
    if($x > $y){
        echo "Giá trị lớn hơn là ".$x;
    }
    else
    {
        echo "Giá trị lớn hơn là ".$y;
    }
?>
```

Câu lệnh: if - elseif - else

- Cú pháp:

if (<biểu thức điều kiện 1>) {

 <các câu lệnh nếu biểu thức điều kiện 1 **đúng**>

}

elseif (<biểu thức điều kiện 2>) {

 <các câu lệnh nếu biểu thức điều kiện 1 **sai** và biểu thức điều kiện 2 **đúng**>

}

else {

 <các câu lệnh nếu biểu thức điều kiện 1 **sai** và biểu thức điều kiện 2 **sai**>

}

Câu lệnh: if - elseif - else

- Ví dụ:

```
<?php
    $x = 15; $y = 10;
    if($x > $y){
        echo "Giá trị lớn hơn là ".$x;
    }
    elseif($x < $y){
        echo "Giá trị lớn hơn là ".$y;
    }
    else{
        echo "Hai giá trị bằng nhau";
    }
?>
```

Câu lệnh: **switch - case**

- Cú pháp: **<?php**

```
switch (<mã trường hợp>) {  
    case <mã 1>:  
        <câu lệnh 1>  
        break;  
    case <mã 2>:  
        <câu lệnh 2>  
        break;  
    case <mã N>:  
        <câu lệnh N>  
        break;  
    default:  
        <câu lệnh mặc định>  
}
```

?>

Câu lệnh: **switch - case**

■ Ví dụ:

```
<?php
    $favcolor = "red";

    switch ($favcolor) {
        case "red":
            echo "Your favorite color is red!";
            break;
        case "blue":
            echo "Your favorite color is blue!";
            break;
        default:
            echo "Your favorite color is neither red nor blue!";
    }
?>
```

Câu lệnh: **while**

- Cú pháp:

```
while (<biểu thức điều kiện>) {  
    <các câu lệnh nếu biểu thức điều kiện đúng>  
}
```

- Lưu ý: Trong vòng lặp **while** phải có câu lệnh thay đổi giá trị của biểu thức điều kiện để tránh việc lặp vô hạn

Câu lệnh: **while**

▪ Nguyên tắc thực thi

(1) Chương trình kiểm tra biểu thức điều kiện

(2) Nếu điều kiện là **true**

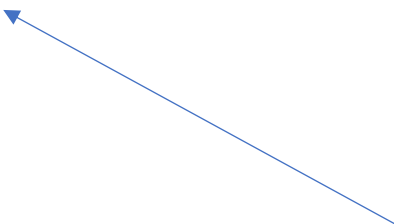
- Thực thi câu lệnh
- Quay lại kiểm tra điều kiện tại bước (2) → Trong câu lệnh thực thi phải có phép toán thay đổi biểu thức điều kiện để chương trình không lặp vô hạn.

(3) Nếu điều kiện **false** chương trình đi đến câu lệnh theo sau cấu trúc **while**

Câu lệnh: **while**

- Ví dụ:

```
<?php
    $x = 1;
    while ($x <= 5) {
        echo "The number is: ".$x."<br>";
        $x++;
    }
?>
```



Thay đổi giá trị trong biểu thức điều kiện

Câu lệnh: **do - while**

- Cú pháp:

```
do {  
    <các câu lệnh>  
} while (<biểu thức điều kiện>)
```

- Lưu ý: Các câu được thực hiện ít nhất một lần. Trong vòng lặp **do - while** phải có câu lệnh thay đổi giá trị của biểu thức điều kiện để tránh việc lặp vô hạn.

Câu lệnh: **do - while**

▪ Nguyên tắc thực thi

(1) Chương trình thực thi ngay **<các câu lệnh>**

- Do đó, **<các câu lệnh>** được thực thi ít nhất 01 lần
- Sau khi thực thi xong, chương trình đánh giá biểu thức điều kiện và kiểm tra

(2) Nếu điều kiện là **true**

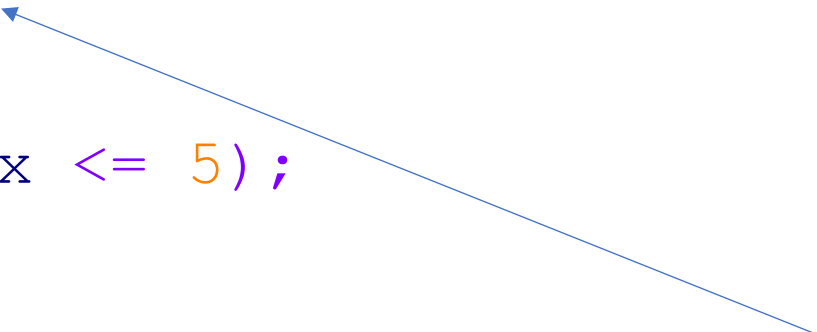
- Đi đến bước (1) để thực thi câu lệnh

(3) Nếu điều kiện **false**: chương trình đi đến câu lệnh theo sau cấu trúc **do while**

Câu lệnh: **do - while**

- Ví dụ:

```
<?php
    $x = 1;
    do {
        echo "The number is: ".$x."<br>";
        $x++;
    } while ($x <= 5);
?>
```



Thay đổi giá trị trong
biểu thức điều kiện

Câu lệnh: **for**

- Cú pháp:

```
for (<khởi tạo>; <điều kiện>; <thay đổi giá trị>) {  
    <câu lệnh 1>  
    <câu lệnh 2>  
    <câu lệnh N>  
}
```

Câu lệnh: **for**

▪ Nguyên tắc thực thi:

(1) Chương trình sẽ khai báo và khởi tạo các biến tại phần <khởi tạo> và kiểm tra biểu thức <điều kiện>

(2) Nếu <điều kiện> là **true**

- Thực hiện câu lệnh trong vòng lặp
- Thực thi lệnh tại phần **<thay đổi giá trị>**
- Kiểm tra lại điều kiện ở Bước (2)

(3) Ngược lại (<điều kiện> là **false**)

- Thực hiện tiếp các câu lệnh phía sau vòng lặp **for**

Câu lệnh: **for**

- Ví dụ:

```
<?php
for ($x = 0; $x <= 10; $x++) {
    echo "The number is: ".$x." <br>";
}
?>
```

```
<?php
for ($x = 0; $x <= 100; $x+=10) {
    echo "The number is: ".$x." <br>";
}
?>
```


Câu lệnh: **foreach**

- Cú pháp dạng 1:

```
foreach ($array as $value)
{
    <câu lệnh 1>
    <câu lệnh 2>
    <câu lệnh N>
}
```

- Cú pháp dạng 2:

```
foreach ($array as $key => $value)
{
    <câu lệnh 1>
    <câu lệnh 2>
    <câu lệnh N>
}
```

Câu lệnh: **foreach**

Cú pháp dạng 1

```
foreach ($array as $value) {  
    <các câu lệnh>  
}
```

▪ Nguyên tắc thực thi:

- (1) Các phần tử trong mảng **\$array** được duyệt theo thứ tự từ phần tử đầu tiên đến phần tử cuối cùng, mỗi lần lặp duyệt một phần tử.
- (2) Tại mỗi lần lặp:
 - + Giá trị của phần tử được duyệt sẽ được gán vào biến **\$value**.
 - + Các câu lệnh trong vòng lặp được thực thi với giá trị của biến **\$value**.
- (3) Vòng lặp dừng khi duyệt qua tất cả phần tử trong mảng **\$array** và thực thi tiếp các câu lệnh phía sau vòng lặp **foreach**.

Câu lệnh: **foreach**

■ Ví dụ:

```
$cars = array("Volvo", "BMW", "Toyota");  
echo $cars[0]; //Volvo  
echo $cars[1]; //BMW  
echo $cars[2]; //Toyota  
  
foreach ($cars as $value) {  
    echo $value."<br/>";  
}
```

Kết quả:

```
Volvo  
BMW  
Toyota
```

Câu lệnh: **foreach**

Cú pháp dạng 2

```
foreach ($array as $key => $value)
{
    <các câu lệnh>
}
```

▪ Nguyên tắc thực thi:

- (1) Các phần tử trong mảng **\$array** được duyệt theo thứ tự từ phần tử đầu tiên đến phần tử cuối cùng, mỗi lần lặp duyệt một phần tử.
- (2) Tại mỗi lần lặp:
 - + Chỉ số hoặc khóa của phần tử được gán vào biến **\$key** và giá trị của phần tử được duyệt sẽ được gán vào biến **\$value**.
 - + Các câu lệnh trong vòng lặp được thực thi với giá trị của biến **\$key** và biến **\$value**.
- (3) Vòng lặp dừng khi duyệt qua tất cả phần tử trong mảng **\$array** và thực thi tiếp các câu lệnh phía sau vòng lặp **foreach**.

Câu lệnh: **foreach**

■ Ví dụ:

```
$employee = array(  
    "Name" => "Alex",  
    "Email" => "alex_jtp@gmail.com",  
    "Age" => 21,  
    "Gender" => "Male");  
  
echo $employee["Name"]. "<br/>"; //Alex  
echo $employee["Age"]; //21  
  
foreach ($employee as $key => $value) {  
    echo $key. ": ".$value. "<br/>";  
}
```

Kết quả:

```
Name: Alex  
Email: alex_jtp@gmail.com  
Age: 21  
Gender: Male
```

Từ khóa **break**

- Khi các câu lệnh trong vòng lặp thực thi đến lệnh **break**, thì vòng lặp sẽ dừng ngay lập tức.
- Ví dụ: Từ khóa **break** trong vòng lặp **for**

```
<?php
    for ($x = 0; $x < 10; $x++) {
        if ($x == 4) {
            break;
        }
        echo "The number is: ".$x."<br>";
    }
?>
```

Từ khóa **break**

- Ví dụ: Từ khóa **break** trong vòng lặp **while**

```
<?php
    $x = 0;
    while ($x < 10) {
        if ($x == 4) {
            break;
        }
        echo "The number is: ".$x." <br>";
        $x++;
    }
?>
```

ĐỊNH NGHĨA HÀM TRONG PHP

- Sử dụng từ khóa **function** để định nghĩa theo cú pháp sau:

```
function functionName() {  
    <các câu lệnh>  
    [return <giá trị trả về>]  
}
```

Ví dụ:

```
<?php  
    function writeMsg() {  
        echo "Hello world!";  
    }  
    writeMsg(); // gọi hàm  
?>
```


ĐỊNH NGHĨA HÀM TRONG PHP

- Định nghĩa hàm có tham số và truyền tham số cho hàm.
- Ví dụ:

```
<?php
```

```
function sum($a, $b) {  
    return $a + $b;  
}
```

```
$a = 5;
```

```
$b = 10;
```

```
$ret = sum($a, $b); // 15
```

```
?>
```

HÀM ĐỊNH NGHĨA SẴN TRONG PHP

- PHP có hơn 1000 hàm định nghĩa sẵn (built-in functions).
- Các hàm có sẵn được gọi trực tiếp và thực hiện một chức năng cụ thể nào đó.

MỘT SỐ HÀM ĐỊNH NGHĨA SẴN TRONG PHP

Tên hàm	Mô tả	Ví dụ
<code>abs(number)</code>	Trị tuyệt đối	<code>echo abs(-1);</code>
<code>date(format, timestamp)</code>	Định dạng ngày giờ	<code>echo date('d-m-Y');</code> <code>echo date('Y');</code>
<code>explode(separator, string, limit)</code>	Tách chuỗi dựa vào <code>separator</code>	<code>\$str = "02/05/2022";</code> <code>\$ret = explode("/", \$str);</code> <code>var_dump(\$ret);</code>
<code>round(number, precision, mode);</code>	Làm tròn	<code>echo round(1.75, 1);</code>
<code>strlen(string)</code>	Trả về độ dài của chuỗi	<code>echo strlen("Hello");</code>
<code>strtolower(string)</code>	Trả về chuỗi in thường	<code>echo strtolower("Hello");</code>
<code>strtoupper(string)</code>	Trả về chuỗi in hoa	<code>echo strtoupper("Hello");</code>
<code>substr(string, start, length)</code>	Trả về chuỗi con	<code>echo substr("Hello", 0, 2);</code>

- Sử dụng cặp thẻ `<?php ?>` để chèn mã PHP vào HTML
- File mã nguồn phải có phần mở rộng là `.php`

`*.php`

```
<!DOCTYPE html>
```

```
<html>
```

```
    <head>
```

```
        <title>Kết hợp HTML và PHP</title>
```

```
    </head>
```

```
    <body>
```

```
        <?php $hk = "I/2022-2023"; ?>
```

```
        <h1>Đại học Ngân hàng TP.HCM</h1>
```

```
        <h1>Học kỳ:<?php echo $hk; ?></h1>
```

```
    </body>
```

```
</html>
```

XỬ LÝ DỮ LIỆU GỬI TỪ FORM HTML

- Tạo form nhập dữ liệu như sau:

Khi nhấn nút **Gửi**, dữ liệu trong form sẽ gửi đến file **age.php** theo phương thức **Post**

```
<form action="age.php" method="post">  
Năm sinh: <input type="text" name="nam_sinh"  
           placeholder = "Nhập năm sinh" /><br/>  
           <input type="submit" value="Gửi">  
</form>
```

Năm sinh:

XỬ LÝ DỮ LIỆU GỬI TỪ FORM HTML

- Tạo file **age.php** để xử lý dữ liệu nhận được.

Lấy dữ liệu được gửi bằng
\$_POST và giá trị thuộc tính **name**

<?php

```
$nam = $_POST["nam_sinh"];  
$tuoi = date('Y') - $nam;  
echo "Tuổi của bạn là: ".$tuoi;
```

?>

age.php

1. Viết chương trình giải phương trình bậc hai $ax^2 + bx + c$. Thiết kế form nhập vào ba hệ số a, b, c
2. Viết chương trình cho biết số nguyên n có phải là số nguyên tố không. Thiết kế form để nhập n .
3. Viết chương trình tìm số lớn nhất trong ba số a, b, c . Thiết kế form nhập a, b, c
4. Viết chương trình tính tổng các số nguyên. Thiết kế form nhập vào các số nguyên, mỗi số ngăn cách với nhau bởi dấu phẩy
5. Viết chương trình kiểm tra ngày có hợp lệ hay không? Thiết kế form để nhập ngày, ngày có định dạng dd/mm/yyyy

HỎI & ĐÁP